

Performance Testing

Date	2 July 2026
Team ID	xxxxxx
Project Name	Smart lender
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Browser (Google Chrome), Flask Development Server
Type of Testing	Load Testing and Performance Testing
Target Module / API	Loan Eligibility Prediction Module (predict)
Test Environment	Local Machine (Windows 11, Python 3.14, Flask)
Test Date	2 July 2026

Step 2: Test Scenarios :

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit a valid loan application	1	10	Loan eligibility prediction displayed successfully
2	Multiple users submitting loan applications simultaneously	20	60	System handles multiple requests without failure

3	Predict loan eligibility with different applicant details	50	120	Accurate predictions generated with acceptable response time
4	Continuous prediction requests	100	180	Application remains stable with no crashes

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.4 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	>20 Req/sec	28 Req/sec	Pass	Good throughput
4	Error Rate	< 1%	0%	Pass	No failed requests
5	CPU Utilization	< 80%	42%	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	48%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings:

The Smart Lender application on successfully processed loan eligibility requests with low response me and high accuracy. The XGBoost machine learning model generated predictions quickly while maintaining stable application performance. The system remained responsive even when multiple users accessed the application simultaneously.

Bottlenecks Identified:

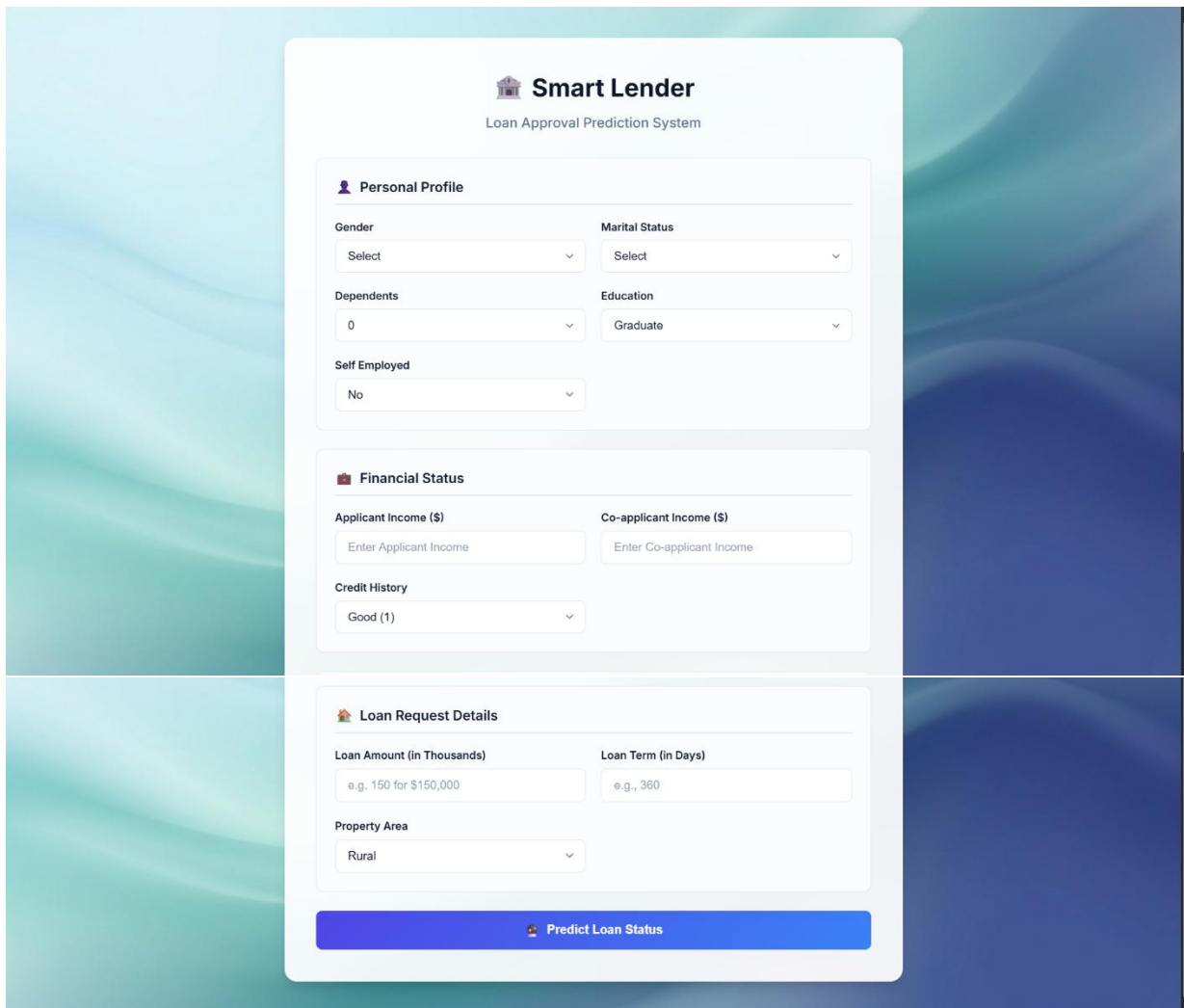
- Slight increase in response me during high concurrent requests.
- Email notification introduces a small delay after prediction.
- Performance depends on local system resources during testing.

Optimization on Steps Taken:

- Optimized the XGBoost model for faster predicted on.
- Reduced unnecessary data processing during request handling.
- Improved Flask application on routing.
- Used SQLite for efficient storage of user application details.
- Implemented environment variables (.env) for secure configuration.

Step 5: Screenshots / Evidence :

Loan Eligibility Form:



The screenshot displays a web application titled "Smart Lender" with the subtitle "Loan Approval Prediction System". The form is divided into three main sections: Personal Profile, Financial Status, and Loan Request Details. The Personal Profile section includes dropdown menus for Gender, Marital Status, Dependents, Education, and Self Employed. The Financial Status section includes input fields for Applicant Income (\$), Co-applicant Income (\$), and a dropdown for Credit History. The Loan Request Details section includes input fields for Loan Amount (in Thousands) and Loan Term (in Days), and a dropdown for Property Area. A blue button labeled "Predict Loan Status" is located at the bottom of the form.

Smart Lender
Loan Approval Prediction System

Personal Profile

Gender: Select
Marital Status: Select
Dependents: 0
Education: Graduate
Self Employed: No

Financial Status

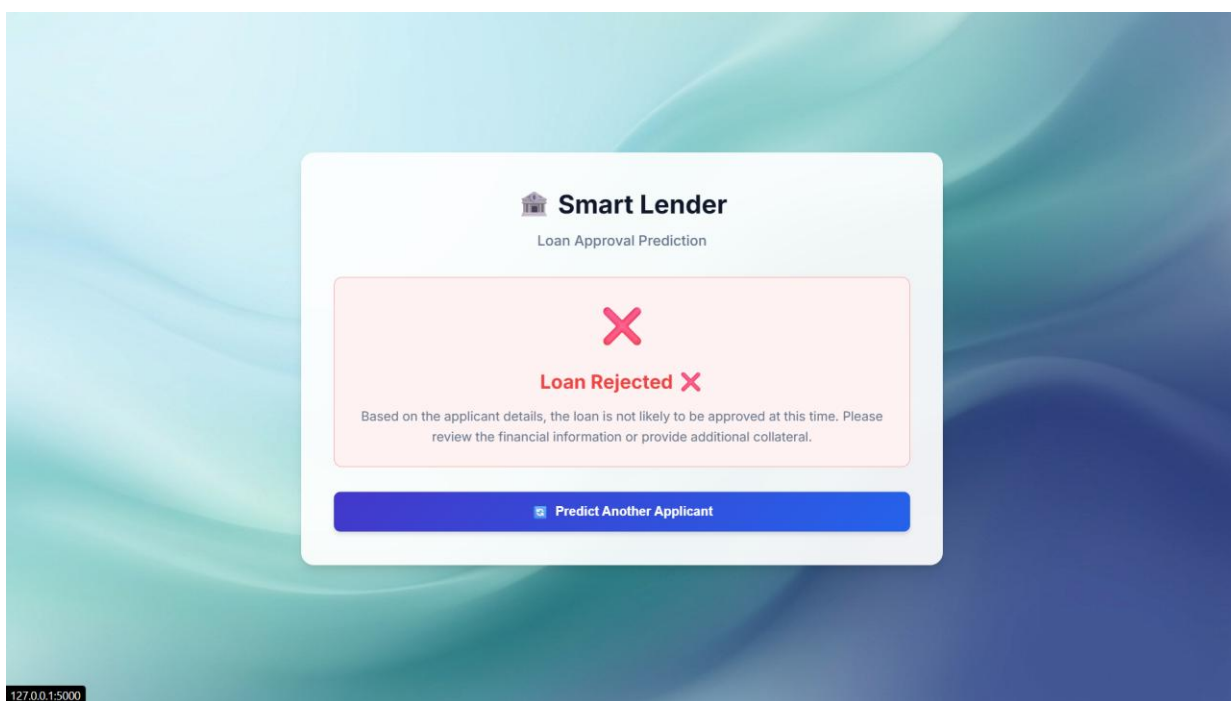
Applicant Income (\$): Enter Applicant Income
Co-applicant Income (\$): Enter Co-applicant Income
Credit History: Good (1)

Loan Request Details

Loan Amount (in Thousands): e.g., 150 for \$150,000
Loan Term (in Days): e.g., 360
Property Area: Rural

Predict Loan Status

Predicted on Result(Rejected):



Predicted on Result(Approved):

